

2026aj05348

September 1, 2026

1 Linear Algebra Assignment

Q1: Solving Linear Systems | Q2: Rank, Covariance & Power Method

BITS ID: 2026AJ05348

All required algorithms are implemented without restricted functions such as `rref`, `matrix_rank`, `solve`, or `eig`. The library function `np.linalg.eigh()` is used only for the comparison required in Q2.3(d).

```
[34]: # Question 1 - Solving Linear Systems

import numpy as np

# Fixed seed makes all random numerical results reproducible.
np.random.seed(42)

TOL = 1e-9
np.set_printoptions(precision=8, suppress=True, linewidth=140)

# =====
# QUESTION 1 - SOLVING LINEAR SYSTEMS
# =====

# -----
# Q1.1 - Convert a matrix to Row Echelon Form (REF)
# No built-in RREF/linear-system solver is used.
# Pivot search is performed manually to avoid division by zero.
# -----

def calculate_ref(M):
    A = M.astype(float).copy()
    rows, cols = A.shape
    pivot_row = 0
    pivot_cols = []

    for col in range(cols):
        if pivot_row >= rows:
            break
```

```

    # Manual search for a suitable pivot.
    pivot = pivot_row
    for r in range(pivot_row + 1, rows):
        if abs(A[r, col]) > abs(A[pivot, col]):
            pivot = r

    # Explicit division-by-zero handling.
    if abs(A[pivot, col]) < TOL:
        continue

    A[[pivot_row, pivot]] = A[[pivot, pivot_row]]

    # Eliminate all entries below the pivot.
    for r in range(pivot_row + 1, rows):
        factor = A[r, col] / A[pivot_row, col]
        A[r, :] = A[r, :] - factor * A[pivot_row, :]

    pivot_cols.append(col)
    pivot_row += 1

    return A, pivot_cols

# -----
# Q1.1 - Convert a matrix to Reduced Row Echelon Form (RREF)
# -----
def calculate_rref(M):
    R, pivot_cols = calculate_ref(M)

    # Work upwards from the last pivot.
    for i in reversed(range(len(pivot_cols))):
        col = pivot_cols[i]
        R[i, :] = R[i, :] / R[i, col]

        # Make all entries above the pivot zero.
        for r in range(i):
            R[r, :] = R[r, :] - R[r, col] * R[i, :]

    return R, pivot_cols

# -----
# Q1.2 - Find pivot/free columns, particular solution and
# null-space basis for  $Ax = b$ .
# -----
def analyse_system(A, b):

```

```

m, n = A.shape

# Construct augmented matrix [A | b].
augmented = np.hstack([A, b.reshape(-1, 1)])

REF, _ = calculate_ref(augmented)
RREF, pivots_all = calculate_rref(augmented)

# Pivot in the final column means inconsistency.
consistent = not any(p == n for p in pivots_all)

# Keep only pivots belonging to A.
pivot_cols = [p for p in pivots_all if p < n]

# All remaining columns are free/non-pivot columns.
free_cols = [j for j in range(n) if j not in pivot_cols]

# Particular solution: set all free variables to zero.
x_p = np.zeros(n)
for i, p in enumerate(pivot_cols):
    x_p[p] = RREF[i, n]

# Construct one null-space basis vector for each free variable.
null_basis = []

for free in free_cols:
    v = np.zeros(n)
    v[free] = 1.0

    for i, p in enumerate(pivot_cols):
        v[p] = -RREF[i, free]

    null_basis.append(v)

return (REF, RREF, pivot_cols, free_cols,
        x_p, null_basis, consistent)

# -----
# Q1.3 - Random 5 x 7 system and complete verification
# -----
A = np.random.randn(5, 7)
b = np.random.randn(5)

REF, RREF, pivot_cols, free_cols, x_p, null_basis, consistent = \
    analyse_system(A, b)

```

```

print("\n===== Q1 =====")
print("A =\n", A)
print("\nb =\n", b)

print("\nAugmented matrix [A | b] =\n",
      np.hstack([A, b.reshape(-1, 1)]))

print("\nREF of [A | b] =\n", REF)
print("\nRREF of [A | b] =\n", RREF)

print("\nConsistent system =", consistent)
print("Pivot columns =", [p + 1 for p in pivot_cols])
print("Non-pivot columns =", [p + 1 for p in free_cols])

print("\nParticular solution x_p =\n", x_p)
print("Check A @ x_p =", A @ x_p)

print("\nNull-space basis:")
for i, v in enumerate(null_basis, 1):
    print("n{} =".format(i), v)
    print("A @ n{} =".format(i), A @ v)

# General solution:
# x = x_p + c1*n1 + c2*n2 + ...
coefficients = np.random.randn(len(null_basis))

x_general = x_p.copy()
for c, v in zip(coefficients, null_basis):
    x_general = x_general + c * v

print("\nRandom coefficients =", coefficients)
print("\nGeneral solution x =\n", x_general)
print("\nA @ x_general =", A @ x_general)
print("Target b =", b)

print("Maximum absolute error =",
      np.max(np.abs(A @ x_general - b)))

```

===== Q1 =====

A =

```

[[ 0.49671415 -0.1382643  0.64768854  1.52302986 -0.23415337 -0.23413696
  1.57921282]
 [ 0.76743473 -0.46947439  0.54256004 -0.46341769 -0.46572975  0.24196227
 -1.91328024]
 [-1.72491783 -0.56228753 -1.01283112  0.31424733 -0.90802408 -1.4123037
  1.46564877]]

```

```

[-0.2257763  0.0675282 -1.42474819 -0.54438272  0.11092259 -1.15099358
0.37569802]
[-0.60063869 -0.29169375 -0.60170661  1.85227818 -0.01349722 -1.05771093
0.82254491]]

```

b =

```

[-1.22084365  0.2088636 -1.95967012 -1.32818605  0.19686124]

```

Augmented matrix [A | b] =

```

[[ 0.49671415 -0.1382643  0.64768854  1.52302986 -0.23415337 -0.23413696
1.57921282 -1.22084365]
[ 0.76743473 -0.46947439  0.54256004 -0.46341769 -0.46572975  0.24196227
-1.91328024  0.2088636 ]
[-1.72491783 -0.56228753 -1.01283112  0.31424733 -0.90802408 -1.4123037
1.46564877 -1.95967012]
[-0.2257763  0.0675282 -1.42474819 -0.54438272  0.11092259 -1.15099358
0.37569802 -1.32818605]
[-0.60063869 -0.29169375 -0.60170661  1.85227818 -0.01349722 -1.05771093
0.82254491  0.19686124]]

```

REF of [A | b] =

```

[[-1.72491783 -0.56228753 -1.01283112  0.31424733 -0.90802408 -1.4123037
1.46564877 -1.95967012]
[ 0.          -0.71964223  0.09194045 -0.32360563 -0.86971955 -0.38638703
-1.26119715 -0.66301499]
[ 0.          0.          -1.27414753 -0.64897609  0.05921705 -1.04190872
-0.0634714  -1.20170441]
[ 0.          -0.          0.          1.91905564  0.40644235 -0.30078496
0.49326694  1.21401792]
[ 0.          0.          0.          0.          -0.45413466 -0.49073933
2.1036828  -2.81197828]]

```

RREF of [A | b] =

```

[[ 1.          -0.          -0.          -0.          -0.          -0.2954518
0.10024944 -1.19899678]
[-0.          1.          -0.          -0.          -0.          -0.4596661
6.69239758 -6.05526114]
[-0.          -0.          1.          -0.          -0.          1.064354
-0.79610146  1.57665988]
[ 0.          -0.          0.          1.          0.          -0.38559999
1.23812212 -0.6787985 ]
[-0.          -0.          -0.          -0.          1.          1.08060313
-4.63228863  6.19194823]]

```

Consistent system = True

Pivot columns = [1, 2, 3, 4, 5]

Non-pivot columns = [6, 7]

```

Particular solution x_p =
[-1.19899678 -6.05526114  1.57665988 -0.6787985   6.19194823  0.          0.]
]
Check A @ x_p = [-1.22084365  0.2088636  -1.95967012 -1.32818605  0.19686124]

Null-space basis:
n1 = [ 0.2954518   0.4596661  -1.064354   0.38559999 -1.08060313  1.
0.          ]
A @ n1 = [-0.  0.  0.  0.  0.]
n2 = [-0.10024944 -6.69239758  0.79610146 -1.23812212  4.63228863  0.
1.          ]
A @ n2 = [-0.  0.  0.  0.  0.]

Random coefficients = [0.73846658 0.17136828]

General solution x =
[-0.99799508 -6.86267776  0.92709656 -0.60622066  6.18778628  0.73846658
0.17136828]

A @ x_general = [-1.22084365  0.2088636  -1.95967012 -1.32818605  0.19686124]
Target b = [-1.22084365  0.2088636  -1.95967012 -1.32818605  0.19686124]
Maximum absolute error = 8.881784197001252e-16

```

```

[35]: # =====
# QUESTION 2 - DATASET, RANK, COVARIANCE AND POWER METHOD
# =====

# -----
# Q2.1 - Generate the 500 x 6 dataset.
# First four features are standard-normal random variables.
# -----

n = 500

f1 = np.random.randn(n)
f2 = np.random.randn(n)
f3 = np.random.randn(n)
f4 = np.random.randn(n)

# Required linear relationships.
f5 = 2 * f1 + 3 * f2
f6 = f3 - 2 * f4

X = np.column_stack([f1, f2, f3, f4, f5, f6])

print("\n===== Q2 =====")
print("\nQ2.1 Dataset")
print("Shape of X =", X.shape)

```

```

print("First 10 rows =\n", X[:10])

# -----
# Q2.2 - Manual rank calculation.
# np.linalg.matrix_rank is NOT used.
# -----
def rank_manual(M):
    A = M.astype(float).copy()
    rows, cols = A.shape
    pivot_row = 0
    rank = 0

    for col in range(cols):
        if pivot_row >= rows:
            break

        # Manual pivot search.
        pivot = pivot_row
        for r in range(pivot_row + 1, rows):
            if abs(A[r, col]) > abs(A[pivot, col]):
                pivot = r

        if abs(A[pivot, col]) < TOL:
            continue

        A[[pivot_row, pivot]] = A[[pivot, pivot_row]]

        # Eliminate entries below the pivot.
        for r in range(pivot_row + 1, rows):
            factor = A[r, col] / A[pivot_row, col]
            A[r, :] = A[r, :] - factor * A[pivot_row, :]

        rank += 1
        pivot_row += 1

    return rank

rank_X = rank_manual(X)

print("\nQ2.2 Rank")
print("Rank(X) =", rank_X)

# -----
# Q2.3(a) - Covariance matrix

```

```

#  $C = (1/n) X^T X$ 
# -----
C = (X.T @ X) / n

print("\nQ2.3(a) Covariance matrix")
print("C =\n", C)

# -----
# Helper functions for the Power Method.
# No np.linalg.norm or eigenvalue solver is used here.
# -----
def vector_norm(v):
    total = 0.0
    for value in v:
        total += value * value
    return total ** 0.5

def matrix_vector_multiply(M, v):
    result = []

    for i in range(len(M)):
        total = 0.0

        for j in range(len(v)):
            total += M[i][j] * v[j]

        result.append(total)

    return np.array(result)

def dot_product(v1, v2):
    total = 0.0

    for i in range(len(v1)):
        total += v1[i] * v2[i]

    return total

# -----
# Q2.3(b) - Power Method.
# The Rayleigh quotient is used for the eigenvalue estimate.
# -----
def power_method(M, tolerance=1e-12):

```



```

v = np.ones(len(M))

norm = vector_norm(v)
for i in range(len(v)):
    v[i] = v[i] / norm

old_lambda = 0.0
iteration = 0

while True:
    iteration += 1

    y = matrix_vector_multiply(M, v)
    norm = vector_norm(y)

    new_v = np.zeros(len(v))
    for i in range(len(v)):
        new_v[i] = y[i] / norm

    Mv = matrix_vector_multiply(M, new_v)
    new_lambda = dot_product(new_v, Mv)

    if abs(new_lambda - old_lambda) < tolerance:
        return new_lambda, new_v, iteration

    v = new_v
    old_lambda = new_lambda

lambda_1, vector_1, iterations_1 = power_method(C)

print("\nQ2.3(b) Power Method")
print("lambda_1 =", lambda_1)
print("v_1 =", vector_1)
print("Iterations =", iterations_1)

# -----
# Q2.3(c) - Deflation
#
#  $C_{new} = C_{current} - v v^T C_{current}$ 
#
# Four non-zero eigenvalues are required because  $\text{rank}(X) = 4$ .
# -----
def outer_product(v):
    return np.outer(v, v)

```

```

def deflate(M, v):
    return M - outer_product(v) @ M

D = deflate(C, vector_1)

power_eigenvalues = [lambda_1]
power_eigenvectors = [vector_1]

for k in range(1, 4):

    lam, vec, it = power_method(D)

    power_eigenvalues.append(lam)
    power_eigenvectors.append(vec)

    print("\nlambda_{} = {:.12f}".format(k + 1, lam))
    print("v_{} = ".format(k + 1), vec)
    print("Iterations =", it)

    D = deflate(D, vec)

# -----
# Q2.3(d) - Library eigenvalue/eigenvector method.
# This is explicitly permitted by the assignment.
# -----
library_values, library_vectors = np.linalg.eigh(C)

# Arrange eigenvalues from largest to smallest.
order = np.argsort(library_values)[::-1]
library_values = library_values[order]
library_vectors = library_vectors[:, order]

print("\nQ2.3(d) Library method")
print("All eigenvalues =", library_values)

for k in range(4):
    print("\nlambda_{} = {:.12f}".format(
        k + 1, library_values[k]))
    print("v_{} = ".format(k + 1),
        library_vectors[:, k])

# -----
# Q2.3(e) - Number of iterations required for accuracy  $10^{-7}$ .

```

```

#
# Accuracy is checked using:
# |estimated eigenvalue - actual eigenvalue| < 10-7
#
# The actual eigenvalues are those obtained in part (d).
# -----
def iterations_for_accuracy(M, actual_lambda, tolerance=1e-7):
    v = np.ones(len(M))

    norm = vector_norm(v)
    for i in range(len(v)):
        v[i] = v[i] / norm

    iteration = 0

    while True:
        iteration += 1

        y = matrix_vector_multiply(M, v)
        norm = vector_norm(y)

        new_v = np.zeros(len(v))
        for i in range(len(v)):
            new_v[i] = y[i] / norm

        Mv = matrix_vector_multiply(M, new_v)
        estimated_lambda = dot_product(new_v, Mv)

        error = abs(estimated_lambda - actual_lambda)

        if error < tolerance:
            return iteration

        v = new_v

D = C.copy()
found_vectors = []

print("\nQ2.3(e) Iterations for accuracy 10-7")

for k in range(4):

    iterations = iterations_for_accuracy(
        D,
        library_values[k],
        1e-7

```

```
)

print("lambda_{}: {} iterations".format(
    k + 1, iterations))

# Deflate using the eigenvector obtained by the Power Method.
_, vec, _ = power_method(D)
found_vectors.append(vec)
D = deflate(D, vec)
```

===== Q2 =====

Q2.1 Dataset

Shape of X = (500, 6)

First 10 rows =

```
[[ -0.11564828   0.52980418   1.07868083   1.66125921   1.35811597  -2.2438376
]
 [ -0.3011037    1.44156862  -0.03850847  -0.45709626   3.72249847   0.87568404]
 [ -1.47852199  -2.4716445   -0.1726273   -0.60221206 -10.37197748   1.03179682]
 [ -0.71984421  -0.79689526   0.88365994   0.46877426  -3.83037418  -0.05388857]
 [ -0.46063877   0.57707213   0.65232288  -0.99838544   0.80993884   2.64909376]
 [  1.05712223  -0.20304539  -1.57639216   0.3017919   1.50510829  -2.17997596]
 [  0.34361829   0.37114587   1.47654035   0.76608028   1.8006742   -0.05562021]
 [ -1.76304016  -0.60398519   1.38009135   1.22693322  -5.33803587  -1.07377509]
 [  0.32408397   0.08658979  -0.6255627   -0.10015408   0.9079373   -0.42525455]
 [ -0.38508228  -0.15567724   0.39580353  -0.20367375  -1.23719627
0.80315103]]
```

Q2.2 Rank

Rank(X) = 4

Q2.3(a) Covariance matrix

C =

```
[[ 0.95192621 -0.04887969 -0.04834925  0.05153219  1.75721334 -0.15141362]
 [-0.04887969  0.97045663  0.05599304 -0.03030361  2.81361051  0.11660026]
 [-0.04834925  0.05599304  1.04637494  0.01437225  0.07128062  1.01763044]
 [ 0.05153219 -0.03030361  0.01437225  0.97204471  0.01215355 -1.92971716]
 [ 1.75721334  2.81361051  0.07128062  0.01215355 11.95525821  0.04697352]
 [-0.15141362  0.11660026  1.01763044 -1.92971716  0.04697352  4.87706475]]
```

Q2.3(b) Power Method

lambda_1 = 12.87665418934814

v_1 = [0.14093412 0.22722351 0.00692585 -0.00014648 0.96353876 0.00721881]

Iterations = 19

lambda_2 = 5.861555859377

```
v_2 = [-0.03689017  0.02191342  0.19216089 -0.35989263 -0.00804007  0.91194614]
Iterations = 10
```

```
lambda_3 = 1.059554587242
v_3 = [-0.38068432  0.25082746  0.79023974  0.40852041 -0.00888626 -0.02680107]
Iterations = 140
```

```
lambda_4 = 0.975360816285
v_4 = [ 0.74036603 -0.49205459  0.41458498  0.19215017  0.00456827  0.03028464]
Iterations = 2
```

Q2.3(d) Library method

```
All eigenvalues = [12.87665419  5.86155586  1.05955459  0.97536082 -0.
-0.          ]
```

```
lambda_1 = 12.876654189348
v_1 = [-0.14093413 -0.2272235  -0.00692582  0.00014642 -0.96353877 -0.00721865]
```

```
lambda_2 = 5.861555859377
v_2 = [ 0.03689014 -0.02191346 -0.19216084  0.35989265  0.0080399  -0.91194614]
```

```
lambda_3 = 1.059554587248
v_3 = [ 0.38069021 -0.25083138 -0.79023645 -0.40851886  0.00888629  0.02680126]
```

```
lambda_4 = 0.975360816280
v_4 = [ 0.74036299 -0.4920526  0.41459128  0.19215341  0.0045682  0.03028445]
```

Q2.3(e) Iterations for accuracy 10^{-7}

```
lambda_1: 11 iterations
```

```
lambda_2: 6 iterations
```

```
lambda_3: 81 iterations
```

```
lambda_4: 1 iterations
```

[]: